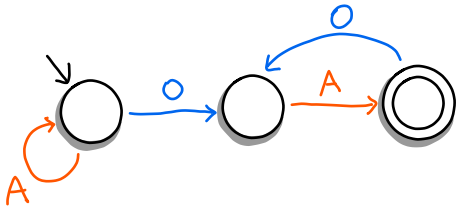
▸ **Aufgabe 1: Dualzahlen** (4 + 4 Punkte)

Die Binärdarstellung einer Zahl x sei 10011.

1. Um welche Zahl handelt es sich, wenn x nach den üblichen Regeln als eine 5-bit-*unsigned-integer*-Zahl kodiert ist? Geben Sie die Zahl x in Dezimalschreibweise an.

2. Um welche Zahl handelt es sich, wenn x nach den üblichen Regeln als 5-bit-*signed-integer*-Zahl kodiert ist? Geben Sie die Zahl x in Dezimalschreibweise an.

▸ **Aufgabe 2: Endliche Automaten** [2 + 2 + 2 + 6 Punkte]
Betrachten Sie folgenden endlichen Automaten:



1. Ist der Automat **deterministisch**?

□ ja □ nein

2. Akzeptiert der Automat den leeren String ε?

□ ja □ nein

3. Akzeptiert der Automat den String "AOOA"?

□ ja □ nein

4. Geben Sie einen **regulären Ausdruck** an, der die Sprache beschreiben, die der Automat akzeptiert.

▸ **Aufgabe 3: Eine kontextfreie Grammatik** [2 + 2 + 2 + 6 Punkte]

Eine kontextfreie Grammatik mit Terminalalphabet T = {a, b} und Nichtterminalsymbolen N = {S, X}, wobei S das Startsymbol ist, habe folgende Produktionsregeln:

$S \rightarrow aSb \mid bX$
$X \rightarrow a \mid bX$

1. Produziert die Grammatik den String "ab"?

□ ja □ nein

2. Produziert die Grammatik den String "abab"?

□ ja □ nein

3. Produziert die Grammatik den String "abba"?

□ ja □ nein

4. Welche Sprache wird von dieser Grammatik produziert? Definieren Sie die Menge L aller von der Grammatik produzierbaren Strings in der Form "L = {…}"!

Übersicht

Extended-Loop-Programmiersprache

⟨Programm⟩ ::= ε | ⟨Befehl⟩ "↲" ⟨Programm⟩
⟨Befehl⟩ ::= "++" ⟨Variable⟩
| "−−" ⟨Variable⟩
| ⟨Variable⟩ "←" ⟨Ausdruck⟩
| ⟨Variable⟩ "[" ⟨Ausdruck⟩ "] ←" ⟨Ausdruck⟩
| "*loop* " ⟨Ausdruck⟩ "↲" ⟨Programm⟩ "*end*"
| "*if* " ⟨Ausdruck⟩ "↲" ⟨Programm⟩ "*end*"
| "*if* " ⟨Ausdruck⟩ "↲" ⟨Programm⟩ "*else*" ⟨Programm⟩ "*end*"
| "*for* " ⟨Variable⟩ "←" ⟨Ausdruck⟩ ", …, " ⟨Ausdruck⟩ "↲" ⟨Programm⟩ "*end*"
| "*return* " ⟨Ausdruck⟩
| "*stop*"
⟨Ausdruck⟩ ::= ⟨Variable⟩ | ⟨Konstante⟩ | ⟨Stringliteral⟩
| "*not*" ⟨Ausdruck⟩
| ⟨Ausdruck⟩ ⟨Operator⟩ ⟨Ausdruck⟩
| "(" ⟨Ausdruck⟩ ")"
| ⟨Variable⟩ "[" ⟨Ausdruck⟩ "]"
| "*len*(" ⟨Variable⟩ ")"
⟨Operator⟩ ::= "+" | "−" | "·" | "/" | "^" | "%" | "*and*" | "*or*" | "=" | "≠" | "<" | ">" | "≤" | "≥"

⟨Variable⟩ : String aus Zahlen und Buchstaben, aber keines der reservierten Worte
⟨Konstante⟩ : Zahl
⟨Stringliteral⟩ : in " " geschriebener String; entspricht einer Zahl

Wenn nicht anders beschrieben, wird die Eingabe in der Variablen *in* übergeben.
"↲" steht für einen Zeilenwechsel.

Extended-While-Programmiersprache

Wie Extended Loop, jedoch zusätzlich mit folgender Produktionsregel:

⟨Befehl⟩ ::= "*while* " ⟨Ausdruck⟩ "↲" ⟨Programm⟩ "*end*"

▸ **Aufgabe 4: Die While-Sprache** [2 + 2 + 2 + 8 Punkte]

Betrachten Sie folgendes Extended-While-Programm, das in der Variablen *in* einen String aus dem Alphabet Σ = {A, B, C} als Eingabe bekommt:

```
n ← len(in)
i ← n−1
while in[i] ≠ "B"
   if in[i] = "A"
      return true
   end
   −−i
end
return false
```

1. Geben Sie ein Beispiel für einen Eingabestring, den das Programm **akzeptiert**, d.h. für den das Programm den Wert *true* zurückgibt!

2. Geben Sie ein Beispiel für einen Eingabestring, den das Programm **ablehnt**, d.h. für den das Programm den Wert *false* zurückgibt!

3. Geben Sie ein Beispiel für einen Eingabestring, bei dem sich das Programm **aufhängt**, d.h. für den die durch das Programm berechnete Funktion *nicht definiert* ist!

4. Schreiben Sie ein Programm in der Programmiersprache **Extended Loop**, das dieselbe Sprache akzeptiert, wie das oben aufgeführte Extended-While-Programm! Beide Programme sollen also für genau dieselben Eingabestrings den Wert *true* zurückgeben.
Achtung: Ihr Programm darf nicht den Befehl "while" enthalten, da dieser Befehl in Extended Loop nicht zur Verfügung steht!

▸ **Aufgabe 5: Dreimal ABBA:** [8 + 6 + 6 Punkte]
Betrachten Sie folgende Sprache:

$L = \{w \in \{a, b\}^* \mid w \text{ enhält den Teilstring "abba"}\}$

Beachten Sie, dass jeder String sich selbst enthält, d.h. "abba" ∈ L.

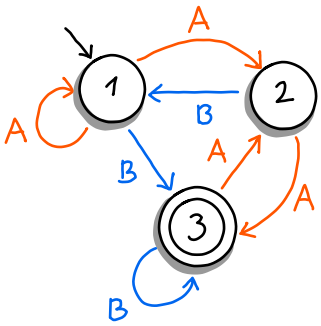
1. Zeichnen Sie einen **Deterministischen** Endlichen Automaten (DEA), der L akzeptiert. Ihr Automat soll also alle Strings akzeptieren, die "abba" enthalten, und umgekehrt müssen alle Strings, die der Automat akzeptiert, "abba" enthalten.
Achtung: Sorgen Sie dafür, dass der Automat auch tatsächlich deterministisch ist!

2. Geben Sie einen **regulären Ausdruck** an, der L beschreibt.

3. Beschreiben Sie eine **kontextfreie Grammatik** (CFG), die L produziert. Die CFG soll also alle Strings und nur die Strings produzieren, die "abba" enthalten. Für die Beschreibung genügt es, alle Produktionsregeln der Grammatik anzugeben. Dabei soll S das Startsymbol sein.

▸ **Aufgabe 6: NEA in DEA verwandeln** [14 Punkte]

Sei Σ = {A, B} ein Alphabet mit zwei Buchstaben. Betrachten Sie folgenden nicht-deterministischen endlichen Automaten (NEA):

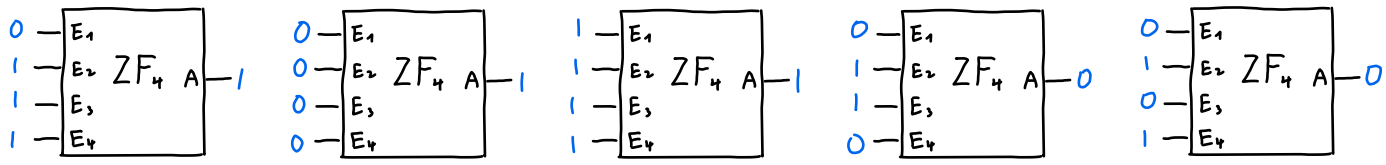


Der NEA hat also drei Zustände: Zustand 1 ist der Startzustand und Zustand 3 ist ein akzeptierender Endzustand.

Verwandeln Sie diesen NEA in einen *deterministischen* endlichen Automaten (DEA). Kennzeichnen Sie in Ihrer Zeichnung den Startzustand und alle akzeptierenden Endzustände des DEA. Beschriften Sie außerdem jeden Zustand Ihres DEA mit der ihm entsprechenden Menge erreichbarer Zustände des NEA. Verschiedene Zustände Ihres DEA müssen dabei mit verschiedenen Teilmengen von {1, 2, 3} beschriftet sein, d.h. Ihr DEA darf insgesamt nicht mehr als $2^3 = 8$ Zustände haben.

▸ **Aufgabe 7: Zeros First** [3 + 3 + 8 + 6 Punkte]

Sei $n \in \mathbb{N}$ mit $n \geq 2$. Wir wollen eine Schaltung ZF_n mit n Eingängen E_1, …, E_n und einem Ausgang A bauen, und es soll gelten: A = 1, genau dann wenn es ein $k \in \{0, \ldots, n\}$ gibt, so dass gilt: $E_i = 0$ für alle $i \leq k$ und $E_j = 1$ für alle $j > k$. Anders ausgedrückt: Wenn im Bitfolge $E_1E_2\ldots E_n$ niemals eine Null auf eine Eins folgt, dann ist A = 1, sonst ist A = 0. Hier sind einige Beispiele für die korrekte Funktion einer ZF_4-Schaltung:



1. Füllen Sie die folgende Wahrheitstabelle für ZF_2 aus:

E_1	E_2	A
0	0	
0	1	
1	0	
1	1	

2. Zeichnen Sie eine Schaltung, die ZF_2 realisiert. In Ihrer Zeichnung dürfen Sie nur die üblichen Gatter für *and*, *or*, *nand* und *nor* − jeweils mit ≥ 2 Eingängen − sowie das *not*-Gatter verwenden.

(Fortsetzung der Aufgabe 7)

3. Angenommen, Sie haben bereits eine Schaltung für ZF_n vorliegen. Wie können Sie damit eine Schaltung für ZF_{2n} mit doppelt so vielen Eingängen realisieren? Zeichnen Sie eine Schaltung für ZF_{2n}; dabei dürfen Sie ZF_n als Baustein verwenden, ohne dabei die Schaltung für ZF_n angeben zu müssen. Außerdem sind in der Schaltung die üblichen Gatter für *and*, *or*, *nand* und *nor* − jeweils mit ≥ 2 Eingängen − sowie das *not*-Gatter erlaubt.

4. Füllen Sie die folgende Tabelle mit Größe und Kosten der Schaltung ZF_n für verschiedene $n \in \mathbb{N}$ aus:

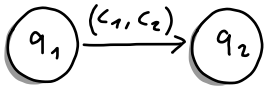
n	Größe von ZF_n	Tiefe von ZF_n
2		
4		
8		
16		

▸ **Aufgabe 8: Zweikopf-Automaten** [14 Punkte]

Ein "endlicher Automat mit zwei Leseköpfen" (2KEA) ist einem ε-NEA (d.h. einem nichtdeterministischen endlichen Automaten mit ε-Übergängen) ähnlich, doch anders als dieser liest er den Eingabestring mit *zwei* unabhängig voneinander arbeitenden Leseköpfen. Beide Leseköpfe arbeiten auf demselben Eingabestring, und wenn der 2KEA startet, zeigen beide Leseköpfe auf das erste Zeichen des Eingabestrings. Die Übergangsfunktion eines 2KEA ist eine Funktion

$$\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q),$$

die wie folgt interpretiert wird: Wenn $q_2 \in \delta(q_1, c_1, c_2)$, dann darf der 2KEA vom Zustand q_1 in den Zustand q_2 überwechseln, wenn er dabei mit Lesekopf 1 das Zeichen c_1 und mit Lesekopf 2 das Zeichen c_2 liest. (Dabei ist Q die Zustandsmenge und Σ das Eingabealphabet. $\mathcal{P}(Q)$ ist die Potenzmenge von Q.) Als Zeichnung können wir das darstellen, indem wir an den Übergang von q_1 nach q_2 das Paar (c_1, c_2) schreiben:



Wenn eines der Zeichen c_1 oder c_2 gleich ε ist, so bedeutet dies, dass der betreffende Lesekopf kein Zeichen lesen soll, sondern auf seiner aktuellen Position verbleibt. Auf diese Weise kann man erreichen, dass die beiden Leseköpfe zum selben Zeitpunkt an verschiedenen Positionen im Eingabestring stehen. Sind c_1 und c_2 gleichzeitig ε, dann ist ein Übergang von q_1 nach q_2 möglich, ohne dass dabei ein Zeichen gelesen wird. Ein 2KEA akzeptiert einen Eingabestring genau dann, wenn es eine Folge von Zuständen $q_1 \to q_2 \to \ldots \to q_n$ vom Startzustand $s = q_1$ zu einem akzeptierenden Endzustand $q_n \in N$ gibt, bei dem jeder Übergang $q_i \to q_{i+1}$ legal nach einer Übergangsregel ausgeführt wird, und am Ende beide Leseköpfe den kompletten Eingabestring gelesen haben. Dabei ist N die Menge der akzeptierenden Endzustände.

Zeichnen Sie einen 2KEA, der die folgende Sprache akzeptiert:

$$L = \{a^n b c^n \mid n \in \mathbb{N}_0\}$$

(Notizen)